

CAD-ANALYZER

Fichas de Extração — LAJES

■ Esta ficha responde: dado um texto/polígono do DXF, qual campo JSON extrair e como.

#	Seção	Conteúdo
1	Identificação	RE_LAJE · RE_LAJE_H · caminhos A e B
2	Contorno e Área	LWPOLYLINE · fórmula Shoelace + diagrama anatômico
3	Espessura h=	extrair h= · range 7–40cm · casos especiais
4	Laje Sintética	clusters h= + diagrama · CLUSTER_RADIUS=500mm
5	Schema JSON Completo	FichaFase3Laje · todos os campos
6	Layers Canônicos	EST-LAJE-TEXT·Painéis·Vazio·encoding CP1252
7	Confidence	fórmula + diagrama · thresholds
8	Aberturas e Recortes	layer Vazio · is_void_layer() · Pilares
9	Exemplos Reais	L5 (1.0) · synth_0 (0.50) · L3 abertura
10	Pipeline Completo	fluxo E2E · checklist 10 passos

```
# Caminho A – ID explícito
RE_LAJE = re.compile(
    r'^(L\d+[A-Za-z]?|Y\d+[A-Za-z]?|X\d+[A-Za-z]?|
    r'|LAJ[-_]? \d+|LAJE[-_]\s*\d+)$',
    re.IGNORECASE
)

# Caminho B – espessura h= (sem ID explícito)
RE_LAJE_H = re.compile(r'h\s*[=:]\s*([\d,.]+)', re.IGNORECASE)
# Ex: 'h=12' 'h = 14' 'h:10' → espessura em cm
```

Texto DXF	RE_LAJE?	RE_LAJE_H?	Resultado
L5	✓	—	codigo="L5"
L12A	✓	—	codigo="L12A"
Y1	✓	—	codigo="Y1"
LAJE 1	✓	—	codigo="LAJE 1"
h=12	—	✓	espessura=12.0cm
L5 h=12	✓	✓	codigo="L5", espessura=12.0cm
L	✗	—	sem número — não casa

Anatomia da Laje no DXF — Todos os Elementos

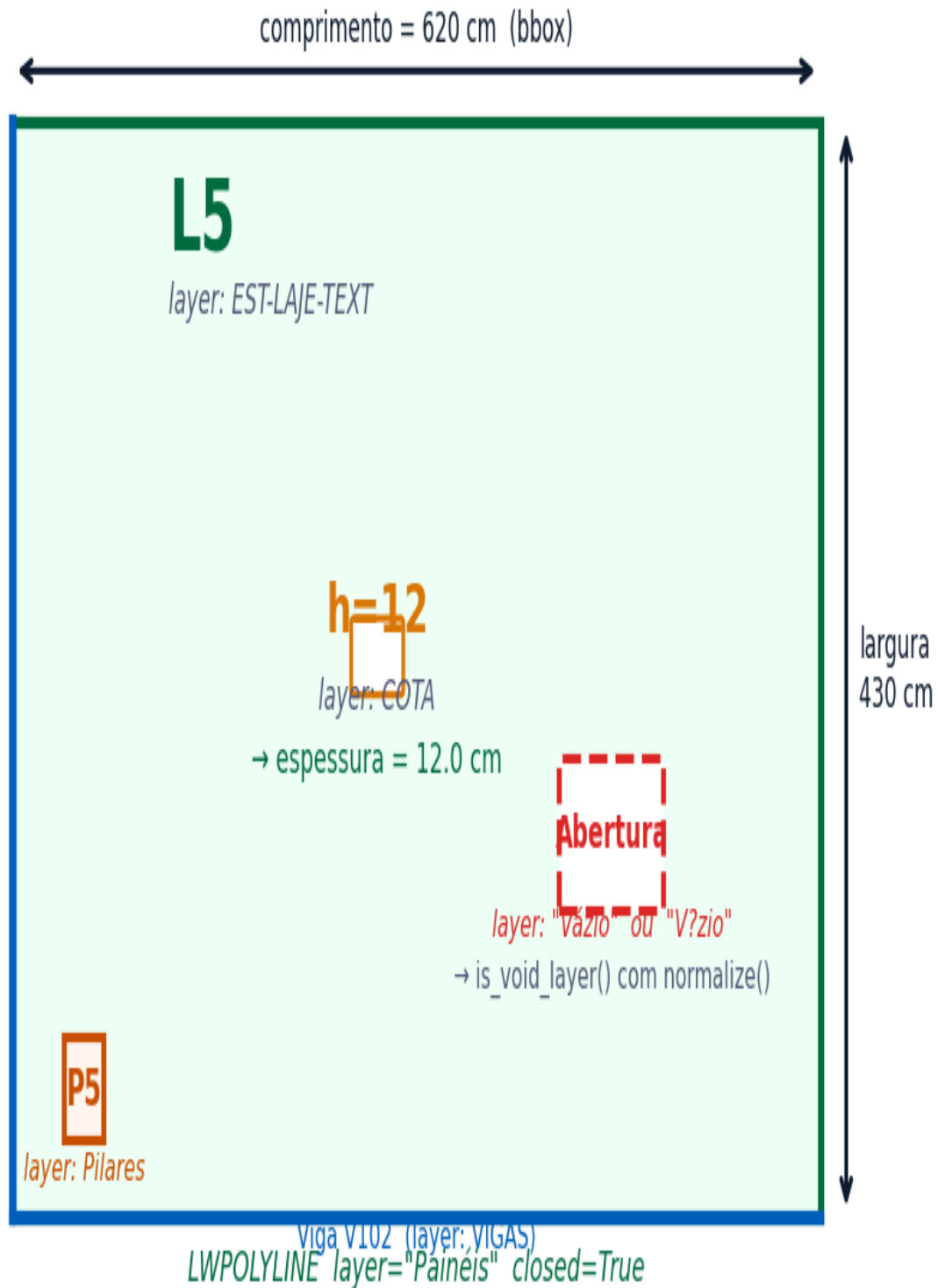


Figura 1 — Anatomia completa da laje: contorno, ID, h=, abertura, recorte pilar, vigas

```
LAJE_SEARCH_RADIUS = 1500.0 # mm

for e in msp.query('LWPOLYLINE'):
    pts = [(float(p[0]),float(p[1])) for p in e.get_points('xy')]
    is_closed = (getattr(e.dxf,'flags',0) & 1 == 1) or e.is_closed
    if is_closed and len(pts) >= 3:
        area = Polygon(pts).area
        if area > 50_000: # mm2 → provável LAJE
            laje_polys.append({'pts':pts,'layer':e.dxf.layer})
        elif area < 5_000: # mm2 → provável PILAR
            pilar_polys.append({'pts':pts,'layer':e.dxf.layer})

# Fórmula Shoelace (área sem shapely):
def area_shoelace(pts):
    n=len(pts); area=0.0
    for i in range(n):
        j=(i+1)%n
        area += pts[i][0]*pts[j][1] - pts[j][0]*pts[i][1]
    return abs(area)/2.0
```

```
def extrair_espessura(textos_h, laje_pos, raio=1500.0):
    lx, ly = laje_pos; candidatos = []
    for t in textos_h:
        m = RE_LAJE_H.search(t['text'])
        if not m: continue
        dist = math.hypot(t['x']-lx, t['y']-ly)
        if dist <= raio:
            val = float(m.group(1).replace(',', '.'))
            candidatos.append((val, dist))
    if candidatos:
        candidatos.sort(key=lambda x: x[1]) # mais próximo
        return candidatos[0][0]
    return 0.0
```

Valor	Válido?	Ação
h=12	SIM (7–40cm)	espessura=12.0
h=5	NÃO (< 7cm)	confidence -= 0.30, log "espessura inválida (norma)"
h=45	NÃO (> 40cm)	confidence -= 0.30, log "espessura fora do range"
ausente	—	espessura=0.0, confidence -= 0.30
2 valores conflitantes	—	usar o mais próximo ao centróide da laje

Laje Sintética — Geração por Clusters de $h=$

Entrada: textos $h=$ sem ID L^*

Saída: Lajes Sintéticas geradas

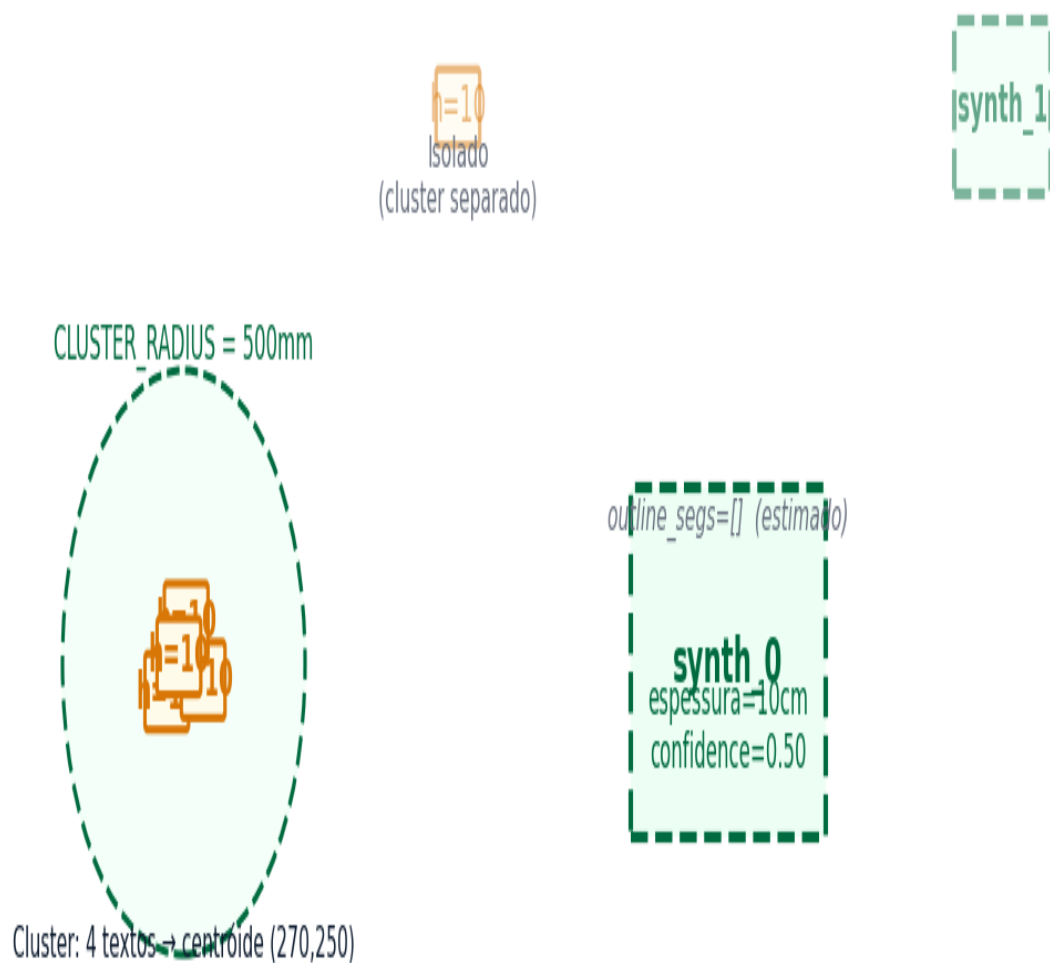


Figura 2 — Múltiplos textos $h=$ dentro de $CLUSTER_RADIUS$ → uma laje SYNTHETIC por cluster

```
CLUSTER_RADIUS = 500.0 # mm

def gerar_lajes_sinteticas(laje_dims):
    used=set(); clusters=[]
    for i,d in enumerate(laje_dims):
        if i in used: continue
        cluster=[d]; used.add(i)
        for j,d2 in enumerate(laje_dims):
            if j in used: continue
            if math.hypot(d['x']-d2['x'],d['y']-d2['y']) < CLUSTER_RADIUS:
                cluster.append(d2); used.add(j)
        clusters.append(cluster)
    result=[]
    for idx,cluster in enumerate(clusters):
        cx=sum(d['x'] for d in cluster)/len(cluster)
        cy=sum(d['y'] for d in cluster)/len(cluster)
        result.append({'id':f'synth_{idx}','name':'SYNTHETIC',
            'x':cx,'y':cy,'espessura':cluster[0]['h_val'],
            'confidence':0.50})
    return result
```

```
{
  "codigo": "L5", # str – ID ("synth_0" se sintética)
  "pavimento": "1_PAVIMENTO", "obra_nome": "ALIMONTI",

  "tipo": "macica", # "macica"|"pre_moldada"|"steel_deck"
  "espessura": 12.0, # float cm – de h=NN

  "dimensoes": {
    "comprimento": 620.0, # cm – bbox do contorno
    "largura": 430.0, # cm – bbox do contorno
    "espessura": 12.0
  },

  "outline_segs": [ # vértices em mm
    {"x":15000.0,"y":10000.0}, {"x":21200.0,"y":10000.0},
    {"x":21200.0,"y":14300.0}, {"x":15000.0,"y":14300.0}
  ],

  "aberturas": [{"pontos":[[5800,5900],[5900,5900],[5900,6100],[5800,6100]],
    "area":20000.0}],

  "vigas_around": ["V101","V102","V103"],
  "pilares_around": ["P5","P6","P7","P8"],

  "nivel": 2.80, "confidence": 0.70, "revisado": false
}
```


✗ ENCODING CRÍTICO: "Vázio" pode chegar como "V?zio" em DXF CP1252. Sempre use `is_void_layer()`.

Canônico	Aliases reais	Uso
SLAB_TEXT	EST-LAJE-TEXT, NOMENCLATURA, EST-TEXT	IDs de lajes (L1, L2...)
PANEL_GEOMETRY	Painéis, PAINEIS, "Pain?is"	Contorno da laje — LWPOLY
PILLAR_CUTOOUT	Pilares, EST-PILAR, EST-PILAR-CUT	Recortes de pilares no contorno
BEAM_INTERFACE_LJ	VIGAS, EST-VIGA	Interface vigas na laje
VOID_OPENING	Vázio, Vazio, "V?zio", ABERTURA, VOID	Aberturas — encoding crítico!
REUSE_STATUS	REAPROVEITAMENTO	BOM/REGULAR/RUIM/DESCARTE

```
VOID_ALIASES = {'vazio', 'vazios', 'abertura', 'aberturas', 'buraco', 'void'}
```

```
def is_void_layer(layer):
    import unicodedata
    n = unicodedata.normalize('NFKD', layer).encode('ascii', 'ignore').decode().lower()
    return n in VOID_ALIASES or 'vaz' in n
```

Confidence da Laje – Composição

Laje SYNTHETIC: sempre confidence = 0.50 (independente da fórmula)

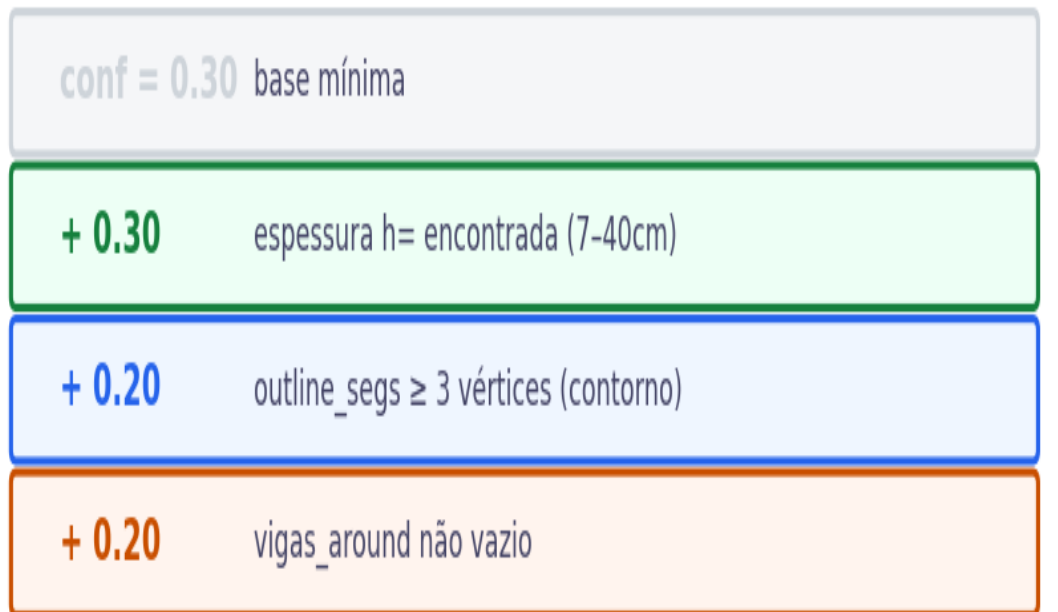


Figura 3 — Composição do score de confidence para lajes

```
def calcular_confidence_laje(laje):  
    conf = 0.30 # base  
    if laje.get('espessura',0) > 0: conf += 0.30  
    if len(laje.get('outline_segs',[])) >= 3: conf += 0.20  
    if laje.get('vigas_around'):  
        conf += 0.20  
    return min(conf, 1.0)  
  
# Laje SYNTHETIC: sempre 0.50 (espessura ok, contorno incerto)
```

```
def detectar_aberturas(laje_contorno, polylines):
    laje_poly = Polygon(laje_contorno)
    aberturas = []
    for poly in polylines:
        if not poly['closed']: continue
        if not is_void_layer(poly['layer']): continue
        ab_poly = Polygon(poly['points'])
        if laje_poly.intersects(ab_poly):
            aberturas.append({'pontos':poly['points'],'area':ab_poly.area})
    return aberturas

def detectar_recortes_pilares(laje_contorno, polylines):
    laje_poly = Polygon(laje_contorno); recortes=[]
    for poly in polylines:
        if not poly['closed']: continue
        ln = normalize_layer(poly['layer'])
        if ln not in ('PILARES','EST-PILAR-CUT','PILLAR-CUT'): continue
        if laje_poly.intersects(Polygon(poly['points'])):
            recortes.append({'pontos':poly['points'],'area':Polygon(poly['points']).area})
    return recortes
```

Exemplo A — L5 (confidence = 1.0)

```
TEXT layer='EST-LAJE-TEXT' text='L5' insert=(18000,12000)
TEXT layer='COTA' text='h=12' insert=(18100,11900)
LWPOLY layer='Paineis' closed=True
  vertices=[(15000,10000),(21200,10000),(21200,14300),(15000,14300)]

# JSON: {"codigo":"L5","espessura":12.0,
#       "dimensoes":{"comprimento":620.0,"largura":430.0,"espessura":12.0},
#       "confidence":1.0}
```

Exemplo B — synth_0 (laje sintética, confidence = 0.50)

```
# 3 textos h=10 dentro de 300mm → 1 cluster → synth_0
# {"codigo":"synth_0","espessura":10.0,"outline_segs":[],"confidence":0.50}
```

Exemplo C — L3 com abertura

```
TEXT layer='EST-LAJE-TEXT' text='L3' insert=(6000,6000)
LWPOLY layer='Vazio' closed=True
  vertices=[(5800,5900),(5900,5900),(5900,6100),(5800,6100)]

# JSON: {"codigo":"L3","aberturas":[{"area":20000.0,...}], "confidence":0.80}
```

Passo	Ação	Detalhe
1	Carregar DXF	ezdxf.readfile(path) → msp
2	normalize_layer()	NFKD → ASCII → UPPER
3	Coletar textos RE_LAJE	TEXT/MTEXT → filtrar → lajes_txt
4	Coletar textos RE_LAJE_H	Todos h=NN → laje_dims
5	Coletar LWPOLYLINE	closed=True, area > 50000mm ² → laje_polys
6	Associar texto→poly	TextAssociator: raio 1500mm → score
7	Extrair espessura h=	extrair_espessura() → mais próximo ao centróide
8	Gerar sintéticas	gerar_lajes_sinteticas() se clusters de h=
9	Detectar aberturas	is_void_layer() + intersects(laje_poly)
10	Calcular confidence	calcular_confidence_laje() → thresholds